29/8 Requerimientos Ágiles y User Stories

1. Fundamentos de la Filosofía Ágil

La **Filosofía Ágil** se presenta no como una metodología o proceso rígido, sino como una **ideología** con un conjunto definido de principios que guían el desarrollo de productos. Es un **compromiso útil entre no tener ningún proceso y tener demasiado proceso**, buscando un equilibrio que evite la burocracia excesiva y la falta de estructura.

¿Qué es Ágil?

- **No es una metodología o proceso**: Ágil es un marco de pensamiento y principios, valores.
- **Es una ideología con principios definidos**: Estos principios guían el desarrollo del producto, enfocándose en la adaptabilidad y la orientación a las personas.

Los equipos ágiles se adhieren a un conjunto de valores que impulsan su forma de trabajar:

- **Planificación continua, multinivel**:
  - La planificación no es un evento único al inicio del proyecto (como en el enfoque tradicional "Big Requirements Up Front" - BRUF), sino un proceso iterativo y adaptable.
  - En Scrum, esto se manifiesta en una **pila de producto** (Product Backlog) viva y priorizada, y en **reuniones de planificación de Sprint** al inicio de cada iteración.
  - En Kanban, la planificación puede ocurrir de manera regular o bajo demanda, desacoplada de la cadencia de entrega.
- **Equipos facultados, auto-organizados, completos**:
  - Los equipos son **auto-organizados** y tienen la autoridad para decidir cómo realizar el trabajo. Las mejores arquitecturas, diseños y requisitos emergen de estos equipos.
  - La auto-organización es clave para reducir los costos de coordinación y fomentar la autonomía individual. Kanban facilita esto al proveer **transparencia del proceso**, metas y objetivos del proyecto, e información sobre riesgos, empoderando a los miembros del equipo para tomar decisiones de priorización y asignación de tiempo.
  - Los equipos suelen ser **interdisciplinarios o multifuncionales**, con todas las habilidades necesarias para completar el trabajo. Aunque Scrum lo prescribe, en Kanban los equipos multifuncionales son opcionales, y los tableros no siempre están ligados a un solo equipo.
- **Entregas frecuentes, iterativas y priorizadas**:
  - El desarrollo es **iterativo**. Se busca entregar software funcional en pequeños incrementos y con alta frecuencia.
  - Las **entregas pequeñas y frecuentes** (como cada dos semanas en Corbis) construyen confianza con los stakeholders, demuestran valor temprano y permiten obtener retroalimentación continua. Esto minimiza el riesgo de desviarse del camino correcto.
  - Los enfoques tradicionales, con entregas grandes y menos frecuentes, a menudo generan mayor incertidumbre y desperdicio.

- **Prácticas de ingeniería disciplinadas**:
  - Ser ágil no es sinónimo de ser indisciplinado. Por el contrario, requiere una alta disciplina técnica para mantener la calidad y el ritmo sostenible.
  - Esto incluye un fuerte **enfoque en la calidad**, ya que los defectos excesivos son el mayor desperdicio en el desarrollo de software.
  - Las prácticas clave incluyen el uso de **ingenieros de pruebas profesionales**, **pruebas unitarias automatizadas**, **inspecciones de código frecuentes**, **análisis y diseño colaborativos**, **patrones de diseño** y **herramientas de desarrollo modernas**.
- **Integración continua (CI)**:
  - La integración frecuente del código es esencial para mantener la calidad y evitar problemas de integración tardíos.
  - Los entornos de CI, a menudo basados en herramientas como Maven y QuickBuild, compilan y ejecutan todas las pruebas automáticamente cada vez que se registran cambios. Esto actúa como un "juez" de la salud del código.
- **Testing Concurrente**:
  - Las pruebas son una parte integral y constante del desarrollo. No son una fase separada y final.
  - Esto se apoya en prácticas como el **Desarrollo Guiado por Pruebas (TDD)** y la inclusión de **encargados de pruebas en los equipos**.
- **Balance entre ningún proceso y demasiado proceso**: La agilidad no elimina el proceso, sino que busca la cantidad justa. Si bien *a menudo se asocia con una menor documentación,* lo más importante es su naturaleza:
  - **Adaptable en lugar de predictivo**: Los métodos ágiles no intentan predecir el futuro lejano con gran detalle, sino que se adaptan a los cambios a medida que ocurren. Esto se alinea con la idea de que los requisitos cambiantes son una ventaja competitiva.
  - **Orientado a la gente en lugar de orientado al proceso**: Prioriza a los individuos y sus interacciones sobre los procesos y las herramientas. Esto fomenta equipos auto-organizados y colaborativos.

Procesos Empíricos - La base del Agilismo.

Los métodos ágiles se basan en la **empirismo**, **lo que significa aprender y adaptarse a través de la experiencia**. Esto se refleja en el ciclo de:

- **Inspección y Adaptación**: Similar al **Kaizen** (mejora continua en terminología Lean), los equipos ágiles cambian algo, observan cómo funciona, aprenden de ello y ajustan su enfoque.
- **Experimentación**: Se diseñan experimentos con la premisa de que es "seguro fallar" (safe-to-fail experiments). Si una hipótesis funciona, el cambio se mantiene; si no, se revierte fácilmente. Kanban, por ejemplo, es un proceso empírico que fomenta el progreso con información imperfecta, observando y ajustando.

2. Valores Ágiles y Principios de los Equipos Ágiles(Revisar, la imagen es mas importante)

📌 1. **Individuos e interacciones por sobre procesos y herramientas**

- **Qué significa:** el éxito de un proyecto depende más de la gente y de cómo se comunica, que de las herramientas o procesos rígidos que use.
- **Ejemplo práctico:** un equipo que se comunica bien en reuniones diarias puede resolver más rápido un bloqueo que si depende de un proceso burocrático con autorizaciones escritas.

📌 2. **Software funcionando por sobre documentación extensiva**

- **Qué significa:** lo esencial es entregar software que funcione y aporte valor al cliente, más allá de tener carpetas de documentación interminable.
- **Importante:** ágil **no elimina la documentación**, sino que la reduce a lo necesario (ejemplo: la arquitectura sí debe quedar documentada).
- **Ejemplo práctico:** un prototipo funcional que ya permita al cliente probar parte de la solución vale más que 50 páginas de especificaciones técnicas.

📌 3. **Colaboración con el cliente por sobre negociación contractual**

- **Qué significa:** el cliente debe ser parte activa del equipo, no alguien que aparece solo al final a reclamar lo que firmó en un contrato. El cliente no debe ser ajeno al proyecto
- **Fricción típica:** cuando ya hay un contrato cerrado con plazos, costos y funcionalidades, y el cliente pide cambios (“me olvidé de algo”). Ahí surge el conflicto entre contrato vs colaboración.
- **Relación con la triple restricción:** si cambian requerimientos, se alteran tiempo, costo o alcance. Un enfoque ágil trata de minimizar esa fricción integrando al cliente día a día.

📌 4. **Responder al cambio por sobre seguir un plan**

- **Qué significa:** los planes son importantes, pero no deben ser “cadenas”. Si el contexto cambia, el equipo debe adaptarse.
- **Ejemplo práctico:** si una startup descubre que su cliente necesita una app móvil en lugar de una web, el equipo ágil ajusta el plan y reorienta el producto.
- **Idea clave:** mejor construir en iteraciones y aprender con el cliente, que seguir un plan rígido hasta el final y entregar algo obsoleto.

Los 12 principios del manifiesto ágil.

1. **Satisfacción del cliente como prioridad.** La meta central es entregar valor de forma temprana y continua.
   - Ejemplo: una app bancaria que entrega primero transferencias y pagos, antes que funcionalidades secundarias.
2. **Aceptar cambios en los requisitos.** Incluso en etapas avanzadas, el cambio no es un problema: es una oportunidad.
   - Ejemplo: un e-commerce que debe adaptarse a nuevos medios de pago digitales que surgen en el mercado.
3. **Entrega frecuente de software funcional.** Cada pocas semanas debe haber algo utilizable.
   - Ejemplo: un equipo Scrum entrega incrementos cada sprint (2–4 semanas).
4. **Colaboración diaria entre negocio y desarrollo.** Los stakeholders no son externos: trabajan codo a codo con los programadores.
   - Ejemplo: el Product Owner participa todos los días en las dailies o revisiones.
5. **Construir proyectos alrededor de individuos motivados.** Un equipo motivado rinde mejor que un equipo con muchas reglas y controles.
   - Ejemplo: Google fomenta equipos pequeños, autónomos y motivados.
6. **La comunicación más eficiente es cara a cara.** Aunque hoy muchas veces es virtual, el principio se refiere a comunicación directa y sin burocracia.
   - Ejemplo: una daily meeting de 15 minutos por videollamada es más efectiva que 10 correos electrónicos.
7. **El software funcionando es la medida principal de progreso.** No son los documentos ni los gráficos de Gantt lo que importa: es que haya entregables que funcionen.
8. **Desarrollo sostenible.** El ritmo debe poder mantenerse indefinidamente. No se trata de “quemar” al equipo con sobrecarga.
   - Ejemplo: iteraciones de 3 semanas constantes en lugar de “maratones” de 80 horas semanales.
9. **Atención continua a la excelencia técnica y al buen diseño.** La calidad no es negociable, porque sin calidad no hay sostenibilidad.
   - Ejemplo: refactorización continua y testing automatizado.

10. **La simplicidad es esencial.** “Menos es más”: evitar agregar funcionalidades que no generan valor real.
    - Ejemplo: no incluir reportes extra en un sistema si el cliente no los pidió.
11. **Equipos auto-organizados.** Las mejores soluciones emergen de equipos que deciden cómo trabajar, en lugar de recibir órdenes externas rígidas.
    - Ejemplo: el equipo decide cómo dividir las historias en tareas, en vez de que un gerente lo imponga.
12. **Reflexión y ajuste continuo.** En intervalos regulares (retrospectivas) el equipo revisa qué puede mejorar y adapta su comportamiento, se retroalimenta regularmente.
    - Ejemplo: después de un sprint, el equipo concluye que necesita mejorar la definición de criterios de aceptación y acuerda hacerlo en el siguiente.

📌 RELACIÓN ENTRE VALORES Y PRINCIPIOS ÁGILES.

1. **Individuos e interacciones sobre procesos y herramientas**

- **Principios vinculados:**
    - Los responsables del negocio y los desarrolladores trabajan juntos cotidianamente (P4).
    - Los proyectos se desarrollan alrededor de individuos motivados (P5).
    - El método más eficiente de comunicación es cara a cara (P6).
    - Equipos auto-organizados generan mejores arquitecturas, requisitos y diseños (P11).
    - A intervalos regulares, el equipo reflexiona y ajusta su comportamiento (P12).
- **Idea central: el éxito depende de las personas, su motivación y comunicación, más que de los procesos o herramientas.**

En un proyecto de e-commerce, los devs y el área de marketing se reúnen a diario por videollamada. El equipo decide por sí mismo cómo dividir historias en tareas, y en cada retrospectiva ajustan cómo organizarse mejor.

---

2. **Software funcionando sobre documentación extensa**

- **Principios vinculados:**
    - Entregar software funcional frecuentemente (P3).
    - El software funcionando es la principal medida de éxito (P7).
    - Los procesos ágiles promueven un desarrollo sostenible (P8).
    - La simplicidad es esencial (P10).
- **Idea central: la prioridad es entregar producto que funcione, liberado en ciclos cortos, asegurando sostenibilidad y evitando burocracia excesiva**.

En un sistema de turnos médicos, el equipo libera cada 3 semanas una versión usable (agenda, cancelaciones, recordatorios), en lugar de esperar meses con toda la documentación lista.

---

3. **Colaborar con el cliente sobre negociación contractual**

- **Principios vinculados:**
    - Nuestra mayor prioridad es satisfacer al cliente (P1).

    - Los responsables de negocio y desarrolladores trabajan juntos (P4).
    - El medio de comunicación por excelencia es cara a cara (P6).
- **Idea central: el cliente es parte del equipo, la confianza y la comunicación superan a lo que pueda estar firmado en un contrato rígido.**

En un sistema contable, el cliente participa en revisiones quincenales, validando pantallas y flujos. Se construye confianza evitando discusiones sobre el contrato original.

---

4. **Responder al cambio sobre seguir un plan**

- **Principios vinculados:**
    - Aceptar que los requisitos cambien, incluso en etapas tardías (P2).
    - Atención continua a la excelencia técnica y al buen diseño (P9).
    - La mejor métrica de progreso es el software funcionando (P7).
- **Idea central: los cambios no son una amenaza, sino una ventaja competitiva. Adaptarse es mejor que seguir un plan fijo que puede quedar obsoleto.**

En un proyecto de app bancaria, a mitad del desarrollo el cliente pide sumar billeteras virtuales. El equipo adapta el backlog y mantiene la calidad con pruebas automatizadas, en lugar de decir “no estaba en el plan inicial”.

---

3. TDD/ATDD y el Desarrollo Ágil de Software

TDD (Test-Driven Development) y ATDD (Acceptance Test-Driven Development)

- **Definición**: TDD implica **escribir un test automático primero**, luego el código mínimo necesario para que ese test pase, y finalmente refactorizar el código. ATDD extiende este concepto a las pruebas de aceptación.
- **Ventajas**:
    - **Mejora la calidad del código**: Al forzar el diseño de código testeable y la detección temprana de errores.
    - **Asegura la cobertura de pruebas**: Se garantiza que el código nuevo esté cubierto por pruebas.
    - **Clarifica requisitos**: La escritura de tests antes del código ayuda a comprender mejor la funcionalidad esperada.
    - **Ayuda en la refactorización**: Las pruebas automatizadas actúan como una red de seguridad, dando confianza para refactorizar y mantener el diseño limpio.
- **Aplicación**: Es una práctica fundamental en el desarrollo ágil y se recomienda para todos los nuevos desarrollos debido a sus grandes beneficios.

4. Frameworks Ágiles

Existen varios **frameworks ágiles** que implementan la filosofía Ágil. Algunos de los mencionados son:

- **FDD (Feature-Driven Development)**: Un enfoque basado en características, con una jerarquía de requisitos (áreas temáticas, conjuntos de características/actividades, características).
- **Crystal**: Una familia de metodologías ágiles desarrolladas por Alistair Cockburn, que se adaptan a diferentes tipos de proyectos y equipos.

- **XP (Extreme Programming)**: Centrado en prácticas de programación para mejorar la calidad y la capacidad de respuesta, como programación en parejas, TDD, diseño incremental e integración continua.
- **Scrum**: Un marco de trabajo para gestionar proyectos complejos de software, que se detalla a continuación.
- **ATDD (Acceptance Test-Driven Development)**: Una práctica que se centra en escribir pruebas de aceptación antes de la implementación, involucrando a los stakeholders.

Comparación Detallada: Scrum vs. Kanban

Scrum y Kanban son dos herramientas de proceso populares en el mundo ágil. Ambas son empíricas y buscan la mejora continua, pero difieren en su enfoque.

Característica	Scrum	Kanban
Naturaleza	Framework/metodología	Método de gestión del cambio
Roles	Prescribe 3 roles: Dueño de Producto, Equipo, Scrum Master.	No establece roles específicos.
Iteraciones/Cadencia	Iteraciones de duración fija (Sprints).	No prescribe iteraciones de tiempo fijo. Cadencia adaptable (regular o bajo demanda).
Límites de WIP	Limita el WIP por iteración (Sprint Backlog).	Limita el WIP por estado en el tablero.
Tablero	Tablero Scrum representa el Sprint Backlog y se reinicia cada Sprint.	Tablero Kanban es persistente, representa un flujo de valor.
Métricas	Gráficos de Burndown.	Lead Time (tiempo de entrega) es la métrica por defecto.
Equipos	Deben ser multidisciplinares.	Pueden ser multidisciplinares o especializados.
Priorización	La pila de producto debe estar priorizada.	La priorización es opcional y puede ser bajo demanda.
Flexibilidad	Más prescriptivo, con más reglas.	Más adaptable, con menos reglas, pero más "mandos que girar".

Puntos clave de la comparación:

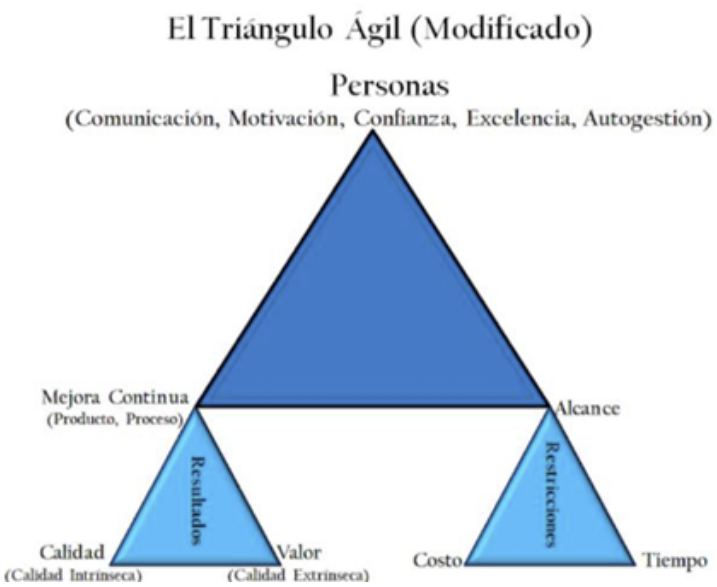
- **Scrum es más restrictivo que Kanban**: Scrum define roles específicos, eventos (planificación de Sprint, Daily Scrum, revisión, retrospectiva) y artefactos (Product Backlog, Sprint Backlog, Incremento) en iteraciones de tiempo fijo.
- **Kanban es un método para la mejora continua**: Se enfoca en la visualización del flujo de trabajo y la limitación del WIP para catalizar el cambio y la mejora evolutiva en un proceso ya existente. El principio de "empezar con lo que haces ahora" es fundamental.
- **Complementariedad**: Ambas herramientas no son mutuamente excluyentes y a menudo se combinan (conocido como **Scrumban**) para aprovechar lo mejor de ambos enfoques. Por ejemplo, equipos Scrum pueden usar paneles Kanban con límites de WIP para gestionar su Sprint Backlog.

5. Ser Ágil no es ser Indisciplinado.

Es un error común asociar la agilidad con la falta de estructura o una "actitud informal". Sin embargo, la filosofía ágil promueve una **disciplina rigurosa**.

- **No es "hacer todo por pedacitos"**: La agilidad no significa trabajar de forma aleatoria o sin orden. Por el contrario, requiere **planificación continua y multinivel**, **entregas frecuentes y priorizadas**, y un conjunto de **prácticas de ingeniería disciplinadas**.

- **Enfoque en la calidad**: La disciplina es fundamental para asegurar la calidad. Como se vio, una de las "recetas para el éxito" en Kanban es precisamente el **enfoque en la calidad**, ya que los defectos excesivos son el mayor desperdicio.
- **Políticas explícitas**: Kanban, por ejemplo, enfatiza la necesidad de **hacer las políticas explícitas** y acordadas por todos para guiar el trabajo y mejorar el flujo.



El Triangulo Agil.

Origen: surge como evolución de la **triple restricción** (alcance–tiempo–costo), que estaba enfocada en el **proyecto**.

Cambio de foco: el triángulo ágil pone en el centro al **producto de software** y a las **personas** que lo construyen.

Ejes principales:

1. **Calidad intrínseca:** características internas del producto (robustez, mantenibilidad, pruebas, diseño correcto).
2. **Calidad extrínseca:** el valor que el producto tiene para el cliente (satisfacción, utilidad real).
3. **Personas:** motivación, confianza y mejora continua del equipo.

Variables de gestión: el alcance, costo y tiempo siguen presentes, pero subordinados al objetivo mayor: **entregar un producto de calidad que evolucione continuamente**.

REQUERIMIENTOS ÁGILES.

Recordemos que la ingeniería de requerimientos tiene **2 procesos**: **por un lado el desarrollo de requerimientos y por otro lado la gestión de los requerimientos.** La gestión está más vinculada con mantener los requerimientos íntegros, bien identificados, bien trazados y que si cambian los requerimientos se actualicen todos los lugares donde se tengan que actualizar esa definición de requerimientos.

> Los requerimientos de negocio y de usuario pertencen al dominio del problema, mientras que los requerimientos de software pertenecen al dominio de la solución.
>
> A nivel de la gestión ágil de requerimientos se trabaja con:
>
> - Requerimiento de negocio
> - Requerimiento de usuario
>
> **Las user stories son requerimientos de usuario alineados a un requerimiento de negocio.**
> **Al usuario le interesa el requerimiento de negocio**. El valor de negocio es el requerimiento de negocio, **pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.**

📌 **Las user stories no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.**

¿Qué debemos asumir?

- **Los cambios son la única constante**. Todo el tiempo va a haber cambios en los requerimientos.
- **Stakeholders: no son todos los que están.** Debemos saber quienes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto. El PO es el representante de todas estas personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- **Siempre se cumple** eso de que: "**El usuario dice lo que quiere cuando recibe lo que pidió".**
- **No hay técnicas ni herramientas que sirvan para todos los casos**. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- **Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de "valor".**

Pilares de los Requerimientos Ágiles.

1. **Construir valor**
   - El objetivo es entregar software que aporte valor real al negocio y satisfaga al cliente.
   - Enfoque: *producto correcto antes que más funciones*.
2. **Sólo lo suficiente**
   - No definir todo de entrada: empezar con lo mínimo y avanzar de forma iterativa.
   - Enfoque: *feedback temprano para evolucionar requerimientos*.
3. **Historias y modelos**
   - Descubrir requerimientos trabajando codo a codo con el cliente.
   - Enfoque: *user stories, comunicación cara a cara, colaboración continua*.

6. Requerimientos Ágiles: El Costo del BRUF y la Adaptabilidad

Como podemos observar en el gráfico, los usuarios terminan usando muy poco de la totalidad de cosas que puede hacer el producto de software. Casi el 80% de las funcionalidades no se usan. Esto es algo muy real y un ejemplo de ello es que por ejemplo, de todo lo que hace excel solo lo usamos para funciones básicas.

La gestión de requisitos es un área donde Agile difiere significativamente de los enfoques tradicionales.

- Usar el “valor” para construir el producto correcto. La prioridad es satisfacer el cliente, por ende la determinacion de los requerimientos van a ser aquellos que le agreguen valor al cliente.
- Usar historias y modelos para mostrar que construir, lenguaje natural para el cliente.
- Determinar que es “solo lo suficiente”

El Costo del Tradicional BRUF (Big Requirements Up Front)

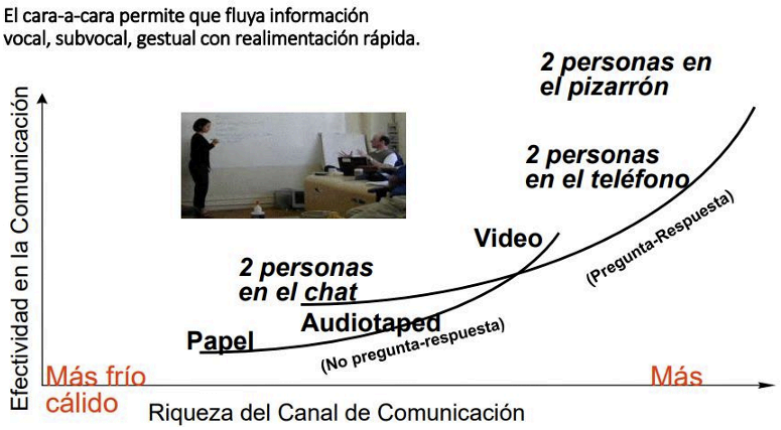
Cuantas funciones tiene una aplicacion que practicamente nadie utiliza, solamente focalizar en lo que va ser utilizable por los usuarios y no features sin sentido.

- **BRUF**: Implica la especificación detallada y exhaustiva de todos los requisitos al inicio del proyecto.
- **Desperdicio significativo**: Jim Johnson de The Standish Group señaló en 2002 que incluso los productos "exitosos" con BRUF tienen un **desperdicio significativo**.
- **Dificultad intrínseca**: Fred Brooks destacó que "la parte más difícil de construir un sistema de software es decidir precisamente qué construir". Un error en la definición temprana de requisitos es muy difícil y costoso de rectificar más adelante.
- **Falta de adaptabilidad**: En un entorno de negocio cambiante, los requisitos iniciales pueden volverse obsoletos antes de que el software sea entregado, generando trabajo desperdiciado.
- **Diferir el análisis detallado**: El principio es **diferir el análisis detallado tan tarde como sea posible**, justo antes de que el trabajo comience. Hasta ese momento, los requisitos se capturan en un formato ligero como las **User Stories**.

Enfoque Just-in-Time:

Los métodos ágiles utilizan un enfoque de requisitos **"justo a tiempo"**, evitando la documentación extensa y detallada al inicio del proyecto.

Es un concepto que se usa fundamentalmente para evitar desperdicios. Y con desperdicio se refiere a especificar requerimientos que después cambian y ya se ha invertido un montón de tiempo en especificarlos.
Por eso postula que se *“analice cuando lo necesite, no antes”*. El producto no va a estar especificado al 100% desde el principio ni muchísimo menos, si no que se van a ir encontrando requerimientos y describiendo los requerimientos que hacen falta conforme haga falta. “ni antes, ni después”.



Mientras más rico el canal de comunicacion mas efectiva es la transmision de la informacion, por eso se favorece la comunicacion directa y personal ya que reduce la AMBIGUEDAD, acelera la retroalimentacion y fortalece la colaboración.

Enfoque TRADICIONAL vs ÁGIL.

- Tradicional = **plan rígido** → fija alcance, ajusta tiempo y recursos.
- Ágil = **valor y adaptación** → fija tiempo y recursos, ajusta alcance.

CONTRASTE ENTRE LA GESTIÓN TRADICIONAL Y EL ENFOQUE ÁGIL

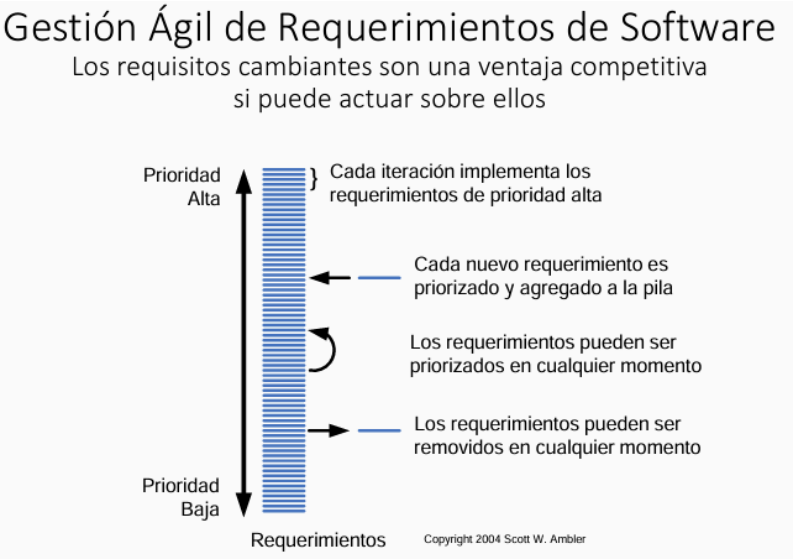
Enfoque tradicional ("dirigido por un plan")

- **Fijo:** los **requisitos** (alcance).
- **Estimado:** los recursos y el tiempo necesarios para cumplir esos requisitos.
- **Lógica:** primero se define *qué se va a hacer* y luego se ajustan costos y calendario para cumplirlo.
- **Problema:** si cambian los requisitos, el plan entero se desestabiliza → sobrecostos, retrasos.

Enfoque ágil ("dirigido por valor")

- **Fijo:** los **recursos** (equipo estable) y el **tiempo** (sprints de duración fija).
- **Variable:** el **alcance** (qué funcionalidades exactas se entregan en cada iteración).
- **Lógica:** en cada sprint se decide junto al Product Owner *qué es lo más valioso para construir* en ese período con el equipo disponible.

- **Ventaja:** se puede ajustar el producto a las necesidades reales del negocio y a los cambios del mercado sin romper la planificación global.



PRODUCT BACKLOG.

El **Product Backlog** es una lista priorizada de todos los requerimientos, funcionalidades, mejoras y correcciones que se deben realizar en el producto.

La **prioridad** se establece en función del **valor de negocio** que cada ítem aporta, y es el **Product Owner** quien define y ordena esta prioridad, manteniendo el backlog actualizado y visible para todo el equipo.

- Prioridad alta - lo que aporta mas valor al cliente, el impacto del cambio es grande si modifico los req de prioridad alta.
- Prioridad Baja - requerimientos wishlist pero que no afectan al valor del cliente
- **Los requisitos cambiantes son una ventaja competitiva**: En lugar de ver los cambios como un problema, Agile los abraza como una oportunidad para obtener una ventaja competitiva, siempre y cuando el equipo pueda actuar sobre ellos rápidamente.

En contraposición con el **enfoque tradicional**, donde **teníamos un usuario** que **no** estaba muy dispuesto a comprometerse y **al no comprometerse en priorizar** dejaba la **responsabilidad de priorizar al equipo que hace el software** y en consecuencia se terminaba **construyendo un software que no dejaba satisfecho al cliente**, ahora **con el enfoque ágil** tenemos **un cliente mucho más partícipe que se encarga de priorizar sus necesidades** (requerimientos).

Con esta situación **se nos permite la compensación de no tener un documento de requerimiento formal escrito detallado con todos los puntos**. Esta gestión ágil lo que ofrece es una compensación al decir que “el mejor medio de comunicación es cara cara y el cliente está disponible para el equipo”.

7. Principios del Manifiesto Ágil aplicados a Requisitos y Colaboración. IMPORTANTE

Cuatro principios del Manifiesto Ágil son particularmente relevantes para la gestión de requisitos y la colaboración:

- **Principio 1: "La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes).”**
  - **Implicación**: La entrega continua de incrementos funcionales genera confianza en el cliente y asegura que el producto evolucione alineado a sus expectativas. Cada release brinda valor

tangible al negocio y permite obtener retroalimentación inmediata.

  - **Comparación**: A diferencia de la gestión tradicional, donde el cliente recién ve el producto terminado al final del proyecto, en Ágil el cliente participa desde el inicio y va validando resultados parciales. Esto reduce riesgos y evita malentendidos.
  - **Ejemplo**: En un sistema de turnos online, en el primer release se entrega la reserva básica de turnos, en el segundo se agrega la cancelación y notificaciones, y en el tercero se integra el pago digital. El cliente percibe valor desde el primer mes.
- **Principio 2: "Recibir cambios de requerimientos, aun en etapas finales."**
  - **Implicación**: La flexibilidad es clave. Los métodos ágiles están diseñados para ser receptivos a los cambios, incluso cuando el proyecto está avanzado. Esto permite que el producto final sea más relevante para las necesidades actuales del mercado y del cliente.
  - **Comparación**: Contradice directamente la gestión de proyectos tradicional, que penaliza los cambios tardíos con sobrecostos y retrasos.
- **Principio 4: "Técnicos y no técnicos trabajando juntos todo el proyecto." MAS IMPORTANTE**
  - **Implicación**: Fomenta una colaboración constante entre los desarrolladores (técnicos) y los stakeholders del negocio (no técnicos). Esta cercanía asegura que el software que se construye está alineado con las expectativas y el valor de negocio.
  - **Ejemplo**: En Kanban, la transparencia del tablero permite a los expertos en la materia ver los efectos de sus acciones y participar activamente en la resolución de impedimentos, mejorando la colaboración.
- **Principio 6: "El medio de comunicación por excelencia es cara a cara." MAS IMPORTANTE**
  - **Implicación**: La comunicación directa y personal es la forma más efectiva y eficiente de transmitir información, especialmente para el conocimiento tácito.
  - **Contexto**: Aunque la documentación escrita existe, el énfasis está en las conversaciones. Las reuniones diarias de pie (Daily Scrums, reuniones frente al tablero Kanban) son ejemplos de esta comunicación frecuente.
  - **Reto en equipos distribuidos**: Para equipos geográficamente dispersos, se deben usar herramientas de videoconferencia y seguimiento electrónico para simular la comunicación cara a cara.
- **Principio 11: "Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados."**
  - **Implicación**: Confía en la capacidad de los equipos para tomar decisiones y diseñar soluciones a medida que el proyecto avanza, en lugar de imponer una arquitectura y diseño predefinidos. No vamos a tener un equipo sin experiencia competente a ver que les sale en el sprint.
  - **Emergencia del diseño**: Esto se alinea con el concepto de **diseño incremental** en XP, donde el diseño evoluciona y mejora continuamente a lo largo del proyecto, en lugar de ser "congelado" al inicio.

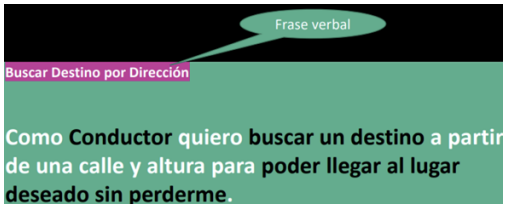
8. User Stories: El Corazón de los Requerimientos Ágiles

¿Qué es una User Story?

Las **User Stories** son la forma principal de capturar los requisitos en muchos entornos ágiles.

- **Contar una historia**: Se les llama "stories" porque la idea es **contar una historia** desde la perspectiva del usuario. Lo escrito en la tarjeta es menos importante que la conversación que genera.

- **Naturaleza Multipropósito**: Una User Story no es solo una descripción, sino un elemento versátil:
  - **Una necesidad del usuario**: Expresan lo que el usuario quiere lograr, enfocándose en el valor de negocio.
  - **Una descripción del producto**: A alto nivel, describe una funcionalidad del sistema.
  - **Un ítem de planificación**: Son unidades de trabajo que pueden ser priorizadas, estimadas y gestionadas en iteraciones.
  - **Un token para una conversación**: Sirven como un recordatorio para futuras discusiones y clarificaciones con el equipo y el Product Owner.
  - **Un mecanismo para diferir una conversación**: Permiten capturar un requisito inicialmente de forma concisa, posponiendo los detalles hasta el momento adecuado, evitando así el BRUF.



➡️ frase verbal
COMO nombre de rol
YO PUEDO actividad
DE TAL FORMA QUE valor del negocio que recibo

10. Estructura de la Historia de Usuario

- `<rol>` **(Who):** quién necesita la funcionalidad.
  - Ejemplo: *cliente registrado*, *administrador de sistema*, *visitante anónimo*.
  - ⚠️ Evitar poner “usuario” genérico, salvo en casos como *logueo de usuario*.
- `<actividad>` **(What):** qué acción quiere realizar.
  - Ejemplo: *consultar mi historial de compras*, *crear una cuenta nueva*.
- `<valor de negocio>` **(Why):** por qué se necesita o qué valor aporta.
  - Ejemplo: *para poder controlar mis gastos mensuales*, *para gestionar eficientemente a mis clientes*.

Ejemplos de User Stories

1. **Como cliente registrado quiero consultar mi historial de compras de forma tal que pueda controlar mis gastos mensuales.**
2. **Como administrador quiero exportar un listado de usuarios en Excel de forma tal que pueda analizar estadísticas de uso.**
3. **Como visitante quiero crear una cuenta de forma tal que pueda acceder a promociones exclusivas.**

- **Intención, no especificación detallada**: Son "expresiones de intención" ("es necesario que haga algo como esto...") y no especificaciones detalladas de requisitos como los casos de uso tradicionales. Se elaboran justo a tiempo, evitando las especificaciones anticipadas.

9. Las Partes de una User Story: Las "Tres C"

Las User Stories se componen tradicionalmente de tres partes interconectadas, conocidas como las "Tres C": **Tarjeta, Conversación y Confirmación**.

- **Tarjeta (Card)**:

- Es la representación física o digital de la historia, usualmente una tarjeta o una entrada en una herramienta de seguimiento.
- Contiene un **resumen conciso** de la historia, a menudo siguiendo el formato "Como , quiero para que ".
- **Ejemplo**: "Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.".

- **Conversación (Conversation)**:
  - Es el **elemento más crítico** de una User Story. La tarjeta es solo un "recordatorio para una conversación futura".
  - Implica **discusiones continuas y colaborativas** entre el equipo de desarrollo, el Product Owner y otras partes interesadas para explorar los detalles de la historia.
  - Estas conversaciones ocurren "tan a menudo como sea necesario" y "con quien sea necesario". Son el lugar donde se aclaran los detalles, se discuten las implicaciones y se llega a un entendimiento compartido.
  - **Principio**: Esto refuerza el principio de **comunicación cara a cara** y el **diferimiento del análisis detallado** hasta el "último momento responsable".
- **Confirmación (Confirmation)**:
  - Representa los **criterios de aceptación** que definen cuándo una historia de usuario se considera completa y satisface la necesidad del usuario.
  - **Propósito**: Ayudan a desarrolladores y testers a **derivar las pruebas** y a los desarrolladores a saber **cuándo dejar de añadir funcionalidad** a la historia. Son esenciales para establecer una definición clara de "terminado".
  - **Características de buenos Criterios de Aceptación**:
    - **Definen una intención, no una solución**: Se enfocan en el *qué* debe hacer el sistema, no en el *cómo*.
    - **Son independientes de la implementación**: No dictan tecnologías o diseños internos.
    - **Relativamente de alto nivel**: Capturan los aspectos clave sin entrar en cada detalle minuto.
  - **Ejemplo**: Para la historia del conductor, los criterios de aceptación podrían ser: "La altura de la calle es un número." y "La búsqueda no puede demorar más de 30 segundos.".

11. User Stories: Porciones Verticales

El concepto de "porciones verticales" en las User Stories es fundamental en el desarrollo ágil. **Debe representar una "rebanada" completa de funcionalidad que atraviesa todas las capas relevantes del sistema, desde la interfaz de usuario hasta la base de datos.**

(un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.)

- **Ejemplo**: Para una historia, se podría trabajar en la **Interfaz Gráfica de Usuario (GUI)**, la **Lógica de Negocio** y la **Base de Datos y el Testing.**

- **Importancia**:
  - **Entrega de valor temprano**: Permite entregar valor real y probado al usuario en cada iteración, ya que cada historia es una funcionalidad completa.
  - **Facilita la integración continua**: Al trabajar en porciones verticales, se reduce la complejidad de la integración tardía, ya que las diferentes capas del sistema se integran y prueban constantemente.
  - **Retroalimentación rápida**: Los stakeholders pueden ver y probar la funcionalidad completa en cada iteración, lo que permite una retroalimentación más rápida y efectiva.
- **Contraste con desarrollo por capas**: Este enfoque contrasta con la forma tradicional de desarrollar por capas (ej., construir toda la capa de GUI, luego toda la lógica de negocio, luego toda la base de datos), lo cual retrasa la entrega de valor y acumula riesgos de integración.

12. Modelado de Roles de Usuario

Comprender a los usuarios es vital para crear User Stories significativas. El **Modelado de Roles** ayuda a construir esta comprensión. **Describimos un perfil de usuario para entenderlo y conocerlo** y para que quien diseña en la **experiencia de usuario**, la diseñe pensando en ese perfil de usuario con cosas concretas.

Ejemplo:

Rol de Usuario: Reclutador Interno
Nos es un experto en computadoras, pero bastante adepto a utilizar la Web. Utilizará el software con poca frecuencia pero muy intensamente. Leerá anuncios de otras compañías para averiguar cuál es la mejor palabra para sus anuncios. La facilidad de uso es importante, pero más importante es que lo que aprenda, lo pueda recordar meses después.

🚫

Tipos de usuarios representantes A EVITAR POR SER GENERICOS:

Si el PO no tiene tiempo de responder nuestras preguntas y formar parte del proyecto activamente, entonces el PO debe elegir a alguien que también conozca el negocio tanto como él y que pueda sernos útil para comunicarnos los requerimientos y necesidades del negocio.
Los usuarios representantes (proxies):
• Gerentes de Usuarios
• Gerentes de Desarrollo
• Alguien del grupo de marketing
• Vendedores
• Expertos del Dominio
• Clientes
• Capacitadores y personal de soporte

- **Propósito**: Crear perfiles detallados de los diferentes tipos de usuarios que interactuarán con el sistema. Esto va más allá de un simple título de "usuario" y busca entender sus características, necesidades y comportamiento.

- **Tarjeta de Rol de Usuario**: Una herramienta para documentar estos perfiles. Contiene información como:
  - **Rol de Usuario**: El nombre o título del rol (ej., "Reclutador Interno").
  - **Experiencia y Habilidades**: Nivel de familiaridad con la tecnología, aptitud para la web, etc. (ej., "No es un experto en computadoras, pero bastante adepto a utilizar la Web").
  - **Frecuencia e Intensidad de Uso**: Cuán a menudo y con qué profundidad utilizará el software (ej., "Utilizará el software con poca frecuencia pero muy intensamente").
  - **Objetivos y Motivaciones**: Qué busca lograr y por qué (ej., "Leerá anuncios de otras compañías para averiguar cuál es la mejor palabra para sus anuncios").
  - **Factores Clave / Preocupaciones**: Aspectos cruciales para su satisfacción o éxito (ej., "La facilidad de uso es importante, pero más importante es que lo que aprenda, lo pueda recordar meses después").
- **Impacto**: Al tener roles bien definidos, se pueden escribir User Stories más precisas y relevantes, asegurando que el desarrollo esté verdaderamente centrado en el usuario y el valor que perciben.

13. Criterios de Aceptación: Confirmando el Valor

Los **criterios de aceptación de la user storie son la base** para que después se construyan los casos de prueba. El criterio de aceptación **es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no**. Debemos saber **si el PO nos va a aceptar esta característica implementada** o no, **porque la va a aceptar si respeta los criterios de aceptación.**

- **Definen límites para una user story (US)**
- **Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos)**
- **Ayudan a que el equipo tenga una visión compartida de la US**
- **Ayudan a desarrolladores y testers a derivar las pruebas.**
- **Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US.**

Definición y Propósito.

- **Guía para pruebas**: Ayudan a desarrolladores y testers a **derivar las pruebas** necesarias para validar la historia.
- **Definición de "Terminado"**: Proporcionan una comprensión clara de **cuándo se ha completado la funcionalidad** y cuándo se puede dejar de añadir más a esa User Story.
- **Evitar malentendidos**: Aseguran que el equipo y el Product Owner compartan una misma visión de lo que se espera de la historia.
- **Ejemplo**: Para la historia "Buscar Destino por Dirección":
  - "La altura de la calle es un número.".
  - "La búsqueda no puede demorar más de 30 segundos.".

Características de Buenos Criterios de Aceptación.

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para llegar al lugar deseado sin perderme.

Criterios de Aceptación:

- **La altura de la calle es un número.**
- **La búsqueda no puede demorar más de 30 segundos.**

- **Definen una intención, no una solución**: Se enfocan en *qué* se debe lograr desde la perspectiva del negocio o del usuario, no en *cómo* se implementará técnicamente.
  - **Ejemplo**: "El usuario debe elegir al menos una cuenta para operar". (Esto es una intención, no dice cómo se implementará la interfaz).
- **Son independientes de la implementación**: No hacen referencia a detalles técnicos internos, arquitecturas o tecnologías específicas.
- **Relativamente de alto nivel**: No es necesario que capturen *cada* detalle diminuto. Se centran en los aspectos clave que confirman que el valor de negocio se ha entregado.

15. Pruebas de aceptación de las US.

Las **Pruebas de Aceptación** son un componente crucial para validar las User Stories y asegurar la calidad.

Rol y Proceso

- **Complementan la User Story**: Estas pruebas **expresan detalles resultantes de la conversación** y confirman que la historia cumple con los criterios de aceptación.
- **Proceso de dos pasos**:
  1. **Identificarlas al dorso de la US**: Los criterios de aceptación se esbozan inicialmente en la tarjeta de la historia.
  2. **Diseñar las pruebas completas**: Una vez que la historia está a punto de ser desarrollada, se diseñan las pruebas de aceptación detalladas. Estas pueden ser:
     - **Manuales**: Realizadas por testers o incluso por el Product Owner.
     - **Automatizadas**: Integradas en el proceso de desarrollo (como en ATDD), proporcionando una red de seguridad para futuras refactorizaciones y cambios.
- **Vinculación con TDD/ATDD**: Estas pruebas son la base de ATDD y se alinean con la práctica de escribir pruebas antes del código, asegurando que el valor de negocio se ha entregado.

¿Cómo escribimos las pruebas de aceptación para la user storie?

Buscar Destino por Dirección	Pruebas de Usuario
Como **Conductor** quiero **buscar un destino** a partir de una calle y altura para **poder llegar al lugar deseado sin perderme**. Criterios de Aceptación: • La altura de la calle es un número. • La búsqueda no puede demorar más de 30 segundos.	❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla). ❑ Probar buscar un destino en un país inexistente (falla). ❑ Probar buscar un destino en País existente, ciudad inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).

14. Gestión de Detalles Adicionales

Una User Story y sus Criterios de Aceptación son deliberadamente de alto nivel. Entonces, ¿**dónde van los detalles** más específicos?

- **Origen de los detalles**: Estos detalles, como el nombre exacto de un encabezado de columna ("Saldo"), el formato de un número ("999.999.999,99") o la elección de un componente de UI (lista desplegable vs. checkbox), surgen de las **conversaciones** continuas y fluidas entre el Product Owner y el equipo de desarrollo.
- **Lugares para capturar los detalles**:
  - **Documentación interna de los equipos**: Esto puede incluir notas de reuniones, diagramas rápidos, comentarios en el código o entradas en wikis internos. La clave es que esta documentación sea ligera y de fácil acceso para el equipo.
  - **Pruebas de aceptación automatizadas**: Estas pruebas son una excelente manera de **expresar los detalles resultantes de la conversación** de forma ejecutable. Al codificar estas pruebas, los detalles se integran en el proceso de desarrollo y se confirman automáticamente con cada integración.
- **Filosofía**: Este enfoque refleja el principio ágil de **diferir el análisis detallado tan tarde como sea posible**, justo antes de que el trabajo de desarrollo comience. Permite que los equipos se mantengan flexibles y respondan a los cambios sin comprometerse a una solución rígida demasiado pronto.

Definition of Ready. DoR.

"Una historia está lista si cumple INVEST".

INVEST model.

Una US es de calidad si cumple con estas 6 condiciones:

1. **Independent (Independiente):**
   - Puede implementarse en cualquier orden.
   - Le da al **PO** la libertad de priorizar sin depender de otras historias.
2. **Negotiable (Negociable):**
   - La US describe el **qué** necesita el usuario, no el **cómo** se implementa.
   - Ejemplo: *"Como cliente quiero pagar con tarjeta"* (no: *"Como cliente quiero un botón azul que se conecte a Visa API"*).
3. **Valuable (Valiosa):**

- Debe tener un **valor de negocio** claro, explicando el *para qué*.
- Esto permite priorizar en el backlog.

4. **Estimatable (Estimable):**
   - Debe ser lo suficientemente clara para poder **estimar esfuerzo/costo**.
   - Sirve para que el cliente compare costo–beneficio.
5. **Small (Pequeña):**
   - Debe poder completarse dentro de **una iteración** (sprint).
   - Nada de historias gigantes que atraviesen varios sprints.
6. **Testable (Comprobable):**
   - Debe tener **criterios de aceptación** verificables.
   - Al final se debe poder probar si la historia fue cumplida o no.

16. Consideraciones Adicionales sobre User Stories

Más allá de su estructura básica, las User Stories tienen implicaciones importantes en la gestión de proyectos y la documentación.

- **No son especificaciones detalladas de requerimientos**: A diferencia de los documentos de requisitos voluminosos o los casos de uso exhaustivos, las User Stories son **ligeras y no buscan ser exhaustivas**. Son un medio para la conversación, no un fin en sí mismas.
- **Son expresiones de intención**: Sirven como un punto de partida para las discusiones, un recordatorio de la necesidad del usuario, encapsulando la idea general ("es necesario que haga algo como esto...").
- **Elaboradas evitando especificaciones anticipadas**: Se detallan en el "último momento responsable", lo que evita el desperdicio del **BRUF** (Big Requirements Up Front) y permite que el desarrollo sea más sensible a los cambios. Esto reduce las demoras en el desarrollo, el inventario de requisitos no utilizados y una definición limitada de la solución.
- **Poco o nulo mantenimiento**: Una vez implementadas y confirmadas, las User Stories a menudo pueden ser "descartadas". El valor se entregó en el software funcional, no en el mantenimiento de la tarjeta.
- **Entrada para documentación incremental**: El código funcionando es la principal "documentación" del sistema. Las User Stories, junto con el código y las pruebas automatizadas, sirven como **entrada para la documentación que se desarrolla incrementalmente** después, no como un paso inicial y pesado.

17. Niveles de Abstracción y Requisitos Jerárquicos

Los requisitos existen en diferentes niveles de detalle, desde ideas de alto nivel hasta especificaciones muy concretas.

- **Jerarquía de la Información**:
  - **Necesidad / Idea / Problema**: Son los conceptos iniciales, las inquietudes o las oportunidades que surgen.
  - **Cambios en el Negocio / Impacto en el Cliente**: La razón fundamental por la que se emprende el trabajo, el valor que se busca entregar.
  - **Ideas / Cambios en el Software**: Las soluciones de alto nivel o funcionalidades que podrían abordar las necesidades.

- **Especificaciones**: Los detalles concretos y funcionales necesarios para implementar el software.

- **Requisitos jerárquicos en proyectos grandes**: En proyectos de mayor envergadura, es común organizar los requisitos de forma jerárquica:
  - **Épicas**: Son User Stories de grano grueso que representan una "unidad atómica de valor para el mercado o el cliente". Son grandes y pueden requerir varias semanas de trabajo. Puede ser un conjunto de funcionalidades relaciones o conjunto de US, como un subconjunto complejo que solo se va desglosar cuando llegue a la cima de la pila del backlog
  - **User Stories**: Son más pequeñas, de grano fino, y se pueden completar en un período de tiempo más corto (idealmente entre medio día y cuatro días de esfuerzo).
  - **Tareas**: El nivel más bajo de detalle, que los desarrolladores usan para gestionar su trabajo diario.
- **Uso en Kanban**: Los equipos pueden rastrear los dos niveles superiores (Épicas y User Stories) en un tablero Kanban, utilizando carriles de nado o colores para diferenciar la jerarquía. Este enfoque de dos niveles permite desacoplar la entrega de valor de la variabilidad del tamaño del ítem de trabajo.

18. Spikes: Manejo de Riesgo e Incertidumbre

Los **Spikes** son un tipo especial de historia utilizada para reducir el riesgo y la incertidumbre en un proyecto ágil.

Definición y Propósito

- **Inversión en investigación**: Un Spike es una historia que tiene como objetivo principal la **investigación o la experimentación** para obtener conocimiento. Debe ser previa a la US relacionada, ya que nos posibilita definir si es viable o no, cuanto tiempo lleva y cuanto puede llegar a costar en tema de recursos (costo y tiempo).
- **Reducir riesgo e incertidumbre**: Permite al equipo explorar una solución antes de comprometerse con su implementación completa, abordando las incógnitas que surgen en el proyecto.
- **Usos comunes**:
  - **Familiarizar al equipo** con una nueva tecnología o un dominio de negocio desconocido.
  - **Analizar una historia compleja** para entenderla mejor y poder dividirla en piezas manejables.
  - **Ganar confianza** frente a riesgos tecnológicos o funcionales, a menudo mediante la investigación o la creación de prototipos.

Tipos de Spikes

1. **Spikes Técnicos**:
   - **Enfoque**: Investigar enfoques técnicos en el dominio de la solución.

     ○ Evaluar performance potencial
     ○ Decisión hacer o comprar
     ○ Evaluar la implementación de cierta tecnología.
   - **Ejemplos**: Evaluar el rendimiento potencial de una tecnología, decidir entre "hacer o comprar" una herramienta, evaluar la implementación de una tecnología específica (ej., determinar requisitos de comunicación o ancho de banda para actualizar un display de cliente).
   - **Objetivo**: Obtener una comprensión más fiable antes de comprometerse con una funcionalidad.
2. **Spikes Funcionales**:
   - **Enfoque**: Investigar cómo interactuará el usuario con el sistema cuando hay incertidumbre.

- **Ejemplos**: Crear prototipos de interfaces de usuario para obtener retroalimentación de los usuarios sobre el diseño y la presentación (ej., crear un prototipo de histograma en un portal web y obtener feedback sobre tamaño y estilo).
- **Objetivo**: Clarificar los requisitos de interacción del usuario y el beneficio esperado.

Lineamientos para Spikes

- **Estimables, demostrables y aceptables**: Aunque son de investigación, deben tener un objetivo claro y un resultado que se pueda verificar.
- **Última opción, no la regla**: Deben reservarse para **incógnitas críticas y grandes**, y no usarse rutinariamente. La excepción es si son muy pequeños y sencillos, en cuyo caso pueden combinarse con la historia resultante en la misma iteración.
- **Iteración separada**: Idealmente, un Spike se implementa en una **iteración separada** de las historias que surgen de él. Esto permite que el aprendizaje del Spike informe la planificación y el desarrollo de las historias reales.

19. Clarificaciones Finales sobre User Stories

Para maximizar la utilidad de las User Stories en el equipo, se debe tener en cuenta:

A. Principios guía

- **Diferir el análisis detallado:** posponé el detalle hasta el "último momento responsable"; capturá antes la intención como US.
- **US ≠ especificación exhaustiva:** son **puntos de partida** para la conversación y la entrega de valor, no documentos completos.
- **La conversación es clave:** la tarjeta recuerda hablar; el entendimiento real nace del intercambio continuo PO–equipo.
- **Perspectiva del usuario:** se escriben desde el **rol que recibe el valor** (who–what–why).

B. Redacción efectiva

- **Una sola pieza de valor:** evitá la palabra **"y"**; si aparece, **dividí** la US.
- **Claridad en lenguaje y foco:** describí el **qué**, no el **cómo** (la implementación se decide luego).
- **Valor explícito:** el "para que…" debe expresar claramente el **beneficio de negocio**.

C. Aceptación y verificación

- **Criterios de aceptación claros y comprobables:** eliminan ambigüedad y orientan pruebas y "Definition of Done".
- **Listas para el sprint:** usá **INVEST** como referencia de calidad y **Definition of Ready** para decidir si entra a la iteración.

D. Antipatrones a evitar

- **Historias gigantes o dependientes** que no puedan terminarse en una iteración.
- **US técnicas puras** que no expresan valor para el usuario/negocio.
- **Forzar todo a US:** si algo no aplica como historia, registralo con otro artefacto (tarea técnica, spike, doc breve).

Checklist rápido antes del Sprint Planning

1. ¿Tiene **rol–actividad–valor**?
2. ¿Está **dividida** (sin “y”)?
3. ¿El **valor de negocio** es claro para priorizar?
4. ¿Cumple **INVEST** y **DoR**?
5. ¿Criterios de aceptación **probables de testear**?
6. ¿Se puede completar en **una iteración**?